Test Strategy

Agree what you test at each level, how much is automated, where the quality gates sit, and what to test hardest.

Engineering Practices · Build Quality In · Inspired by the Test Pyramid (Mike Cohn)

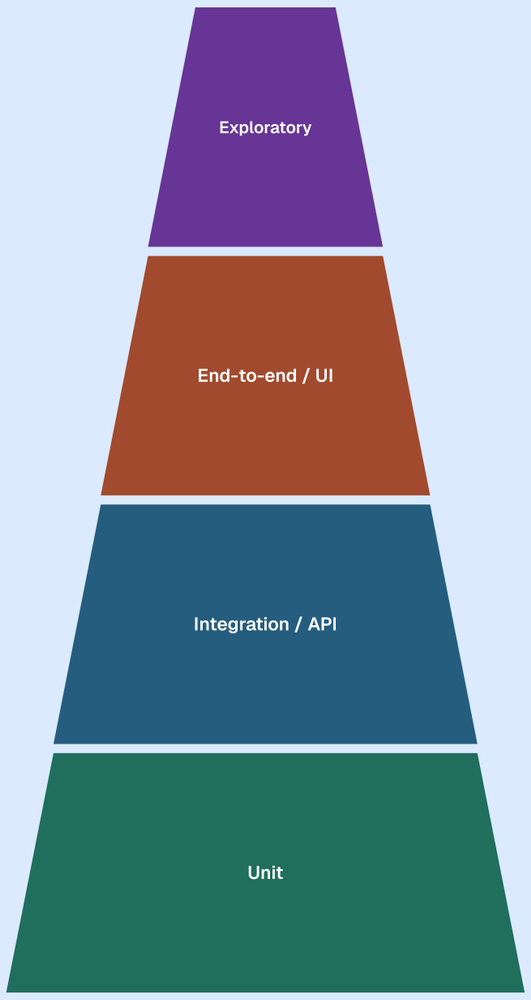
WORKED EXAMPLE · FICTIONAL TEAM

TEAM / PRODUCT: Basket web app

DATE: 16 May

1 Test Pyramid

MANY FAST TESTS AT THE BOTTOM, FEW SLOW ONES AT THE TOP



EXPLORATORY
People testing what scripts can't

What we test: reorder journeys on real phones
Automated: no
Runs when: each story, before release
Owner: Alex

END-TO-END / UI
Whole journeys, through the interface

What we test: reorder, checkout, substitutes
Automated: yes: 12 journeys
Runs when: every merge to main
Owner: Alex, Ravi

INTEGRATION / API
Services and components together

What we test: basket API, prices, stock replica
Automated: yes
Runs when: every commit
Owner: developers

UNIT
Small pieces of code, in isolation

What we test: price rules, basket maths
Automated: yes: 1,400 tests
Runs when: every commit
Owner: developers

2 Quality Gates

WHAT MUST PASS, AND WHERE IN THE PIPELINE

WHERE	CHECK	BLOCKS RELEASE?	OWNER
Commit	unit tests pass	yes	developer
Pull request	review + integration tests	yes	reviewer
Main	end-to-end journeys	yes	Alex
Main	security scan: no high risks	yes	Platform
Staging	accessibility check (WCAG AA)	yes	Mei
Staging	price and promo tests	yes	Ravi
Release	self-service checks	yes	team

3 Shift Left

FIND PROBLEMS EARLIER

- [x] Examples agreed before coding
- [x] Testers join refinement
- [x] Tests written with the code
- [x] Accessibility checked early
- [x] Security checked early
- [ ] Performance tested early
- [ ] Test data on demand ← copy of real orders

4 Test Hardest

HIGHEST RISK

- Promo prices after a promotion ends
- Payments and refunds
- Substitutes: right item, right price
- Screen reader on checkout

5 Our Definition of Done Says

THE QUALITY BAR FOR EVERY ITEM

tests pass at every level, reviewed by a second person, accessible (WCAG AA), deployed to staging, and accepted by Priya

6 Test Debt

FLAKY, SLOW OR MISSING TESTS

Fix first: 6 flaky reorder tests (Ravi)

Time set aside each iteration: half a day, from the 20% for debt

"How would we know, within minutes, that we'd broken something?"

"Which bug would hurt our users most, and are we testing for it?"

© 2026 Eric Parker · CC BY 4.0 · eparker71.github.io/agile-ways-of-working